PHP

Object-Oriented Programming

for Web Development

Using LAMPP and Visual Studio Code

Topics Covered:

- ✔️ Classes & Objects
- ✔️ Constructors & Destructors
- ✔️ Encapsulation, Inheritance & Polymorphism
- ✔️ OOP Database Connections (MySQLi & PDO)
- ✔️ Form Handling & CRUD Applications
- ✔️ Mini Project: Student Management System
- ✔️ 30+ Interview Questions with Answers

Chapter 1: Introduction to PHP & Object-Oriented Programming

1.1 What is PHP?

PHP (Hypertext Preprocessor) is a popular open-source, server-side scripting language designed primarily for web development. It is embedded in HTML and runs on the server before the page is sent to the user's browser. PHP is fast, flexible, and widely used — it powers more than 77% of websites on the internet, including platforms like WordPress, Facebook (early versions), and Wikipedia.

PHP files have the extension .php and are executed on a web server such as Apache (provided by XAMPP or LAMPP).

Key Facts About PHP

Created by: Rasmus Lerdorf (1994)

Current Version: PHP 8.x

File Extension: .php

Server: Apache, Nginx (via XAMPP/LAMPP)

Database Integration: MySQL, PostgreSQL, SQLite

Platform: Windows, Linux, macOS

1.2 What is Object-Oriented Programming (OOP)?

Object-Oriented Programming (OOP) is a programming approach that organizes code into reusable units called objects. Instead of writing long lists of instructions (procedural style), OOP groups related data and functions together into classes.

Think of OOP like the real world: a Car is an object. It has properties (color, speed, model) and behaviors (drive, stop, honk). In PHP OOP, we model these real-world concepts directly in our code.

1.3 Four Pillars of OOP

Pillar	Simple Definition	Real-World Analogy
Encapsulation	Hide internal data; expose only what is needed	A car engine — you press the pedal, not rewire the engine
Inheritance	A class can reuse properties/methods of another class	A child inherits traits from a parent
Polymorphism	Same method name, different behavior in different classes	A 'speak' command — a dog barks, a cat meows
Abstraction	Show only essential features, hide complexity	A TV remote — you press ON, the internal circuit is hidden

1.4 Why Use OOP in Web Development?

Modern web applications are complex. They need to manage users, products, orders, payments, and much more. OOP makes this manageable by:

- Organizing code into logical, reusable blocks
- Making large projects easier to maintain and update
- Reducing code duplication through inheritance
- Improving security through encapsulation (data hiding)
- Making teamwork easier — different developers can work on different classes

1.5 Procedural Programming vs OOP

Procedural Programming	Object-Oriented Programming (OOP)
Code is a sequence of instructions	Code is organized into classes and objects
Data and functions are separate	Data (properties) and functions (methods) are together
Difficult to reuse code	Code is easily reused through inheritance
Hard to maintain in large projects	Easy to maintain and extend
No data hiding	Data can be hidden using encapsulation
Example: basic PHP scripts	Example: Laravel, Symfony frameworks

1.6 Setting Up Your Environment

Installing XAMPP (Windows/macOS)

1. Download XAMPP from https://www.apachefriends.org
2. Run the installer and select Apache + MySQL + PHP
3. Start XAMPP Control Panel and click Start next to Apache and MySQL
4. Place your PHP project in C:\xampp\htdocs\
5. Open browser and go to http://localhost/yourproject/

Installing LAMPP (Linux/Ubuntu)

```
# Download LAMPP installer
sudo chmod +x xampp-linux-x64-8.x.x-installer.run
sudo ./xampp-linux-x64-8.x.x-installer.run

# Start LAMPP
sudo /opt/lampp/lampp start
```

```
# Place files in:
/opt/lampp/htdocs/yourproject/
```

Setting Up Visual Studio Code

6. Download VS Code from https://code.visualstudio.com
7. Install the PHP Intelephense extension (better code completion)
8. Install PHP Debug extension (for debugging)
9. Install Prettier or PHP CS Fixer for code formatting

1.7 Your First PHP OOP Script

Let's verify your setup works with a simple PHP script:

```php
<?php
// File: index.php
// Place in: htdocs/test/index.php

class Greeting {
    public string $message;

    public function __construct(string $name) {
        $this->message = "Hello, $name! Welcome to PHP OOP.";
    }

    public function display(): void {
        echo $this->message;
    }
}

$greet = new Greeting("Alice");
$greet->display();
// Output: Hello, Alice! Welcome to PHP OOP.
?>
```

Open http://localhost/test/ in your browser. You should see the greeting message.

📋 **Chapter Summary**

- PHP is a server-side scripting language used for web development.
- OOP organizes code into classes and objects instead of sequential instructions.
- The four pillars of OOP are: Encapsulation, Inheritance, Polymorphism, Abstraction.
- OOP is preferred for large web projects due to reusability and maintainability.
- XAMPP/LAMPP provides Apache + PHP + MySQL for local development.
- VS Code with PHP extensions is the recommended editor.

Chapter 2: Classes and Objects

2.1 What is a Class?

A class is a blueprint or template for creating objects. It defines the structure — what data an object will hold (properties) and what actions it can perform (methods). Think of a class like a cookie cutter: the cutter is the class, and the cookies are the objects.

> **Definition**
> A class is a user-defined data type that contains properties (variables) and methods (functions).
>
> A class does not use memory until an object is created from it.

2.2 What is an Object?

An object is an instance of a class. When you create an object from a class, you are creating a specific copy of that blueprint with its own data. Multiple objects can be created from the same class.

2.3 Basic Class Syntax

```
<?php
class ClassName {
    // Properties (variables)
    public $propertyName;

    // Methods (functions)
    public function methodName() {
        // code here
    }
}

// Creating an object
$object = new ClassName();
?>
```

2.4 The $this Keyword

Inside a class method, $this refers to the current object. It is used to access the object's own properties and methods.

```
<?php
class Car {
    public $brand = "Toyota";
```

```
    public function getBrand() {
        return $this->brand;  // $this refers to the current object
    }
}
$car = new Car();
echo $car->getBrand();  // Output: Toyota
?>
```

2.5 Student Class — Full Example

Let's build a complete Student class with properties and methods:

```
<?php
class Student {
    // Properties
    public string  $name;
    public int     $age;
    public string  $course;
    private string $studentId;

    // Constructor (covered in Chapter 3)
    public function __construct(string $name, int $age, string $course) {
        $this->name      = $name;
        $this->age       = $age;
        $this->course    = $course;
        $this->studentId = "STU-" . rand(1000, 9999);
    }

    // Method: Register the student
    public function register(): string {
        return "Student {$this->name} has been registered for {$this->course}.";
    }

    // Method: Display info
    public function displayInfo(): void {
        echo "=== Student Information ===\n";
        echo "Name    : {$this->name}\n";
        echo "Age     : {$this->age}\n";
        echo "Course  : {$this->course}\n";
        echo "ID      : {$this->studentId}\n";
    }
}

// ─── Creating Objects ──────────────────────────────
$student1 = new Student("Alice Mwangi", 20, "Computer Science");
$student2 = new Student("Bob Ochieng", 22, "Information Technology");

echo $student1->register();
```

```
// Output: Student Alice Mwangi has been registered for Computer Science.

$student1->displayInfo();
$student2->displayInfo();
?>
```

2.6 Properties in Detail

Access Modifier	Meaning
public	Accessible from anywhere — inside/outside the class
private	Only accessible inside the class that defines it
protected	Accessible inside the class and its child classes

2.7 Multiple Objects from One Class

```
<?php
class Book {
    public string $title;
    public string $author;
    public float  $price;

    public function __construct(string $title, string $author, float $price) {
        $this->title  = $title;
        $this->author = $author;
        $this->price  = $price;
    }

    public function getSummary(): string {
        return "\"{$this->title}\" by {$this->author} — $" . number_format($this-
>price, 2);
    }
}

$book1 = new Book("PHP & MySQL", "Robin Nixon", 39.99);
$book2 = new Book("Clean Code", "Robert Martin", 34.95);
$book3 = new Book("Design Patterns", "Gang of Four", 49.00);

echo $book1->getSummary() . "\n";
echo $book2->getSummary() . "\n";
echo $book3->getSummary() . "\n";
?>
```

2.8 Static Properties and Methods

Static members belong to the class itself, not to any specific object. They are accessed using the :: operator and the self keyword.

```
<?php
class Counter {
    public static int $count = 0;

    public static function increment(): void {
        self::$count++;
    }

    public static function getCount(): int {
        return self::$count;
    }
}

Counter::increment();
Counter::increment();
Counter::increment();
echo Counter::getCount();  // Output: 3
?>
```

📋 Chapter Summary

- A class is a blueprint; an object is an instance (copy) of that class.
- Properties store data; methods define behavior.
- Use $this to refer to the current object inside a class.
- Access modifiers: public (open), private (class-only), protected (class + children).
- Multiple objects can be created from one class, each with its own data.
- Static members belong to the class, not to individual objects.

✏️ Exercise: Classes and Objects

1. Create a Car class with properties: brand, model, year, color. Add a method describe() that prints all details.

2. Create 3 different Car objects and call describe() on each.

3. Add a static property $totalCars to count how many Car objects have been created.

4. Create a Product class with: name, price, quantity. Add a method totalValue() that returns price × quantity.

5. Create a BankAccount class with: owner, balance. Add methods deposit() and withdraw() that update the balance.

Chapter 3: Constructors

3.1 What is a Constructor?

A constructor is a special method that is automatically called when an object is created. It is used to initialize the object's properties with starting values. You do not call a constructor manually — PHP calls it for you when you use the new keyword.

Key Points

Constructor method name: __construct()

Called automatically when: $obj = new ClassName()

Purpose: Initialize object properties

Can accept parameters for custom initialization

3.2 Syntax

```
<?php
class ClassName {
    public function __construct(/* optional parameters */) {
        // Initialization code runs automatically
        $this->property = value;
    }
}
?>
```

3.3 Student Constructor Example

```
<?php
class Student {
    public string $name;
    public int    $age;
    public string $course;

    // Constructor
    public function __construct(string $name, int $age, string $course) {
        $this->name   = $name;
        $this->age    = $age;
        $this->course = $course;
        echo "Student object created for: {$name}\n";
    }

    public function getInfo(): string {
        return "{$this->name} | Age: {$this->age} | Course: {$this->course}";
    }
```

```
}

// Object creation triggers the constructor automatically
$s1 = new Student("Alice", 20, "Computer Science");
// Output: Student object created for: Alice

echo $s1->getInfo();
// Output: Alice | Age: 20 | Course: Computer Science
?>
```

3.4 User Constructor Example

```
<?php
class User {
    public string  $username;
    public string  $email;
    private string $password;
    public string  $role;
    public string  $createdAt;

    public function __construct(
        string $username,
        string $email,
        string $password,
        string $role = "user"   // Default parameter
    ) {
        $this->username  = $username;
        $this->email     = $email;
        $this->password  = password_hash($password, PASSWORD_BCRYPT);
        $this->role      = $role;
        $this->createdAt = date("Y-m-d H:i:s");
    }

    public function getProfile(): void {
        echo "Username : {$this->username}\n";
        echo "Email    : {$this->email}\n";
        echo "Role     : {$this->role}\n";
        echo "Created  : {$this->createdAt}\n";
    }
}

$admin = new User("admin", "admin@school.com", "pass123", "admin");
$user1 = new User("jdoe", "john@email.com", "secret");

$admin->getProfile();
?>
```

3.5 Product Constructor Example

```
<?php
class Product {
    public string $name;
    public float  $price;
    public int    $stock;
    public string $category;

    public function __construct(
        string $name,
        float  $price,
        int    $stock,
        string $category = "General"
    ) {
        $this->name     = $name;
        $this->price    = $price;
        $this->stock    = $stock;
        $this->category = $category;
    }

    public function display(): void {
        echo "[{$this->category}] {$this->name} — $" . number_format($this->price,
2);
        echo " | Stock: {$this->stock}\n";
    }

    public function isAvailable(): bool {
        return $this->stock > 0;
    }
}

$p1 = new Product("Laptop", 850.00, 15, "Electronics");
$p2 = new Product("PHP Book", 29.99, 50, "Books");
$p3 = new Product("USB Cable", 4.99, 0);

$p1->display();
echo $p3->isAvailable() ? "In stock" : "Out of stock";  // Out of stock
?>
```

3.6 Constructor Property Promotion (PHP 8+)

PHP 8 introduced a shorthand syntax to declare and initialize properties directly in the constructor parameter list:

```
<?php
// Traditional way (PHP 7):
class Vehicle {
    public string $brand;
```

```
    public int    $year;
    public function __construct(string $brand, int $year) {
        $this->brand = $brand;
        $this->year  = $year;
    }
}

// PHP 8 shorthand (Constructor Property Promotion):
class VehicleModern {
    public function __construct(
        public string $brand,
        public int    $year
    ) {}  // Properties are declared and initialized automatically!
}

$car = new VehicleModern("Toyota", 2023);
echo $car->brand;  // Toyota
?>
```

📋 Chapter Summary

- __construct() is automatically called when an object is created with new.
- Constructors initialize object properties with starting values.
- Constructors can have required parameters and optional (default) parameters.
- PHP 8 constructor property promotion reduces boilerplate code.
- Password hashing (password_hash) should always be done in the constructor for User classes.

Chapter 4: Destructors

4.1 What is a Destructor?

A destructor is the opposite of a constructor. It is a special method called automatically when an object is destroyed or when the script ends. Its main purpose is to clean up resources such as database connections, file handles, or memory.

Key Points

Destructor method name: __destruct()

Called automatically when: object goes out of scope, unset($obj), or script ends

Purpose: Release resources (close DB connections, close files)

Cannot have parameters

4.2 Basic Destructor Example

```
<?php
class FileLogger {
    private $fileHandle;
    private string $filename;

    public function __construct(string $filename) {
        $this->filename   = $filename;
        $this->fileHandle = fopen($filename, "a");
        echo "Logger opened: {$filename}\n";
    }

    public function log(string $message): void {
        fwrite($this->fileHandle, date("[Y-m-d H:i:s] ") . $message . "\n");
    }

    // Destructor — automatically closes the file
    public function __destruct() {
        fclose($this->fileHandle);
        echo "Logger closed: {$this->filename}\n";
    }
}

$logger = new FileLogger("app.log");
$logger->log("Application started");
$logger->log("User logged in");
// Script ends — __destruct() runs automatically
// Output:
// Logger opened: app.log
// Logger closed: app.log
?>
```

4.3 Object Lifecycle

Phase	Description
1. Creation	new ClassName() — __construct() is called
2. Usage	Methods are called, properties are accessed
3. Destruction	unset($obj) or end of scope — __destruct() is called
4. Cleanup	PHP garbage collector frees the memory

4.4 Database Connection Destructor

```
<?php
class DatabaseConnection {
    private mysqli $connection;

    public function __construct(
        string $host = "localhost",
        string $user = "root",
        string $pass = "",
        string $db   = "school"
    ) {
        $this->connection = new mysqli($host, $user, $pass, $db);
        if ($this->connection->connect_error) {
            die("Connection failed: " . $this->connection->connect_error);
        }
        echo "Database connected.\n";
    }

    public function query(string $sql): mixed {
        return $this->connection->query($sql);
    }

    // Automatically closes when object is destroyed
    public function __destruct() {
        $this->connection->close();
        echo "Database connection closed.\n";
    }
}

$db = new DatabaseConnection();
// Use $db->query(...) for operations
// Connection closes automatically at end of script
?>
```

📋 Chapter Summary

- __destruct() is automatically called when an object is destroyed or script ends.
- Destructors are used to free resources: close DB connections, close file handles.
- Destructors cannot accept parameters.
- PHP automatically calls destructors during garbage collection.
- Good practice: Always release resources in the destructor to prevent memory leaks.

Chapter 5: Encapsulation

5.1 What is Encapsulation?

Encapsulation is the practice of bundling data (properties) and the methods that operate on that data within a class, and restricting direct access to some of the object's components. It is like putting your valuables in a safe — you control who can access them and how.

Encapsulation protects data from accidental modification and provides a controlled interface to interact with the object's internal state.

5.2 Access Modifiers

Modifier	Accessible Inside Class?	Accessible in Child Class?	Accessible Outside Class?
public	Yes	Yes	Yes
protected	Yes	Yes	No
private	Yes	No	No

5.3 Getters and Setters

When a property is private, you use getter and setter methods to read and write its value. This allows validation and control:

```
<?php
class BankAccount {
    private string $owner;
    private float  $balance;

    public function __construct(string $owner, float $initialBalance = 0.0) {
        $this->owner   = $owner;
        $this->balance = $initialBalance;
    }

    // Getter — read the balance
    public function getBalance(): float {
        return $this->balance;
    }

    // Getter — read the owner
    public function getOwner(): string {
        return $this->owner;
    }

    // No direct setter for balance — controlled through methods
    public function deposit(float $amount): void {
        if ($amount <= 0) {
```

```
            throw new InvalidArgumentException("Deposit must be positive.");
        }
        $this->balance += $amount;
    }

    public function withdraw(float $amount): void {
        if ($amount > $this->balance) {
            throw new Exception("Insufficient funds.");
        }
        $this->balance -= $amount;
    }
}

$account = new BankAccount("Alice", 1000.00);
$account->deposit(500);
$account->withdraw(200);
echo "Balance: $" . $account->getBalance();  // Balance: $1300

// This would cause an ERROR — balance is private:
// $account->balance = 999999;  // Fatal error!
?>
```

5.4 User Account System — Full Encapsulation Example

```
<?php
class UserAccount {
    private string $username;
    private string $email;
    private string $passwordHash;
    private string $role;
    private bool   $isActive;

    public function __construct(
        string $username, string $email, string $password, string $role = 'user'
    ) {
        $this->setUsername($username);
        $this->setEmail($email);
        $this->setPassword($password);
        $this->role     = $role;
        $this->isActive = true;
    }

    // Setters with validation
    public function setUsername(string $username): void {
        if (strlen($username) < 3) {
            throw new InvalidArgumentException("Username too short (min 3
chars).");
        }
        $this->username = $username;
```

```
    }

    public function setEmail(string $email): void {
        if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            throw new InvalidArgumentException("Invalid email address.");
        }
        $this->email = $email;
    }

    public function setPassword(string $password): void {
        if (strlen($password) < 8) {
            throw new InvalidArgumentException("Password must be 8+ characters.");
        }
        $this->passwordHash = password_hash($password, PASSWORD_BCRYPT);
    }

    //Getters
    public function getUsername(): string { return $this->username; }
    public function getEmail(): string    { return $this->email; }
    public function getRole(): string     { return $this->role; }
    public function isActive(): bool      { return $this->isActive; }

    // Business methods
    public function verifyPassword(string $password): bool {
        return password_verify($password, $this->passwordHash);
    }

    public function deactivate(): void { $this->isActive = false; }
    public function activate(): void   { $this->isActive = true; }
    // NOTE: Password hash is NEVER exposed via getter — true encapsulation!
}

$user = new UserAccount("alice99", "alice@example.com", "mypassword");
echo $user->getUsername() . "\n";   // alice99
var_dump($user->verifyPassword("mypassword"));  // bool(true)
?>
```

📋 Chapter Summary

- Encapsulation hides internal data and provides controlled access via methods.
- Use private for sensitive data (passwords, balances); public for open data.
- Getters read private/protected properties; setters write them with validation.
- Encapsulation prevents invalid data from being assigned directly.
- The password hash is a classic example never expose it via a getter.

✏️ Exercise: Encapsulation

1. Create a Temperature class with a private celsius property. Add getters/setters that also

provide Fahrenheit conversion.

2. Create a Student class where grade (0–100) is private with a setter that rejects values outside that range.

3. Extend the UserAccount class above with a lastLogin property that updates automatically when verifyPassword() succeeds.

Chapter 6: Inheritance

6.1 What is Inheritance?

Inheritance allows a class (child) to acquire the properties and methods of another class (parent). This promotes code reuse you write common code once in the parent class, and all child classes automatically get it.

In PHP, inheritance is achieved using the extends keyword.

Syntax

class ChildClass extends ParentClass { ... }

The child class inherits all public and protected properties and methods from the parent.

The child class can add new properties/methods and override existing ones.

6.2 Person → Student / Lecturer / Employee

```
<?php
// Parent Class
class Person {
    protected string $name;
    protected int    $age;
    protected string $email;

    public function __construct(string $name, int $age, string $email) {
        $this->name  = $name;
        $this->age   = $age;
        $this->email = $email;
    }

    public function introduce(): string {
        return "Hi, I am {$this->name}, age {$this->age}.";
    }

    public function getEmail(): string { return $this->email; }
}

// Child Class: Student
class Student extends Person {
    private string $course;
    private string $studentId;

    public function __construct(string $name, int $age, string $email, string
$course) {
        parent::__construct($name, $age, $email);  // Call parent constructor
```

```
        $this->course    = $course;
        $this->studentId = "STU-" . rand(1000, 9999);
    }

    public function enroll(): string {
        return "{$this->name} is enrolled in {$this->course} (ID: {$this-
>studentId}).";
    }
}

// Child Class: Lecturer
class Lecturer extends Person {
    private string $department;
    private string $qualification;

    public function __construct(
        string $name, int $age, string $email,
        string $department, string $qualification
    ) {
        parent::__construct($name, $age, $email);
        $this->department    = $department;
        $this->qualification = $qualification;
    }

    public function teach(): string {
        return "{$this->name} ({$this->qualification}) teaches in {$this-
>department}.";
    }
}

// Child Class: Employee
class Employee extends Person {
    private string $position;
    private float  $salary;

    public function __construct(
        string $name, int $age, string $email, string $position, float $salary
    ) {
        parent::__construct($name, $age, $email);
        $this->position = $position;
        $this->salary   = $salary;
    }

    public function getPaySlip(): string {
        return "{$this->name} | {$this->position} | Salary: $" .
number_format($this->salary, 2);
    }
}

//Usage
$s = new Student("Alice", 20, "alice@uni.edu", "Computer Science");
```

```
$l = new Lecturer("Dr. John", 45, "john@uni.edu", "ICT", "PhD");
$e = new Employee("Mary", 32, "mary@uni.edu", "Accountant", 3500.00);

echo $s->introduce() . "\n";  // Inherited from Person
echo $s->enroll()    . "\n";  // Student-specific
echo $l->teach()     . "\n";
echo $e->getPaySlip(). "\n";
?>
```

6.3 The parent:: Keyword

When a child class has a constructor, you should call parent::__construct() to also run the parent constructor and initialize inherited properties. The parent:: keyword is also used to call overridden methods from the parent.

6.4 Method Overriding

A child class can redefine a method from the parent class. This is called overriding. The child version replaces the parent's version for that class.

```
<?php
class Animal {
    public string $name;
    public function __construct(string $name) { $this->name = $name; }

    public function speak(): string {
        return "{$this->name} makes a sound.";
    }
}

class Dog extends Animal {
    public function speak(): string {  // Overrides parent method
        return "{$this->name} says: Woof!";
    }
}

class Cat extends Animal {
    public function speak(): string {  // Overrides parent method
        return "{$this->name} says: Meow!";
    }
}

$dog = new Dog("Rex");
$cat = new Cat("Whiskers");
echo $dog->speak();  // Rex says: Woof!
echo $cat->speak();  // Whiskers says: Meow!
?>
```

6.5 The final Keyword

Use final on a class to prevent it from being extended, or on a method to prevent it from being overridden.

```
<?php
class Base {
    final public function coreMethod(): string {
        return "This cannot be overridden.";
    }
}

final class Sealed {
    // This class cannot be extended
}
?>
```

📋 Chapter Summary

- Inheritance lets a child class reuse code from a parent class using extends.
- Child classes inherit all public and protected members of the parent.
- Always call parent::__construct() in child constructors to initialize parent properties.
- Method overriding allows a child class to replace a parent method's behavior.
- Use final to prevent further inheritance or method overriding.
- PHP supports single inheritance only (one parent class per child).

Chapter 7: Polymorphism

7.1 What is Polymorphism?

Polymorphism (from Greek: 'many forms') means that the same method name can produce different behaviors in different classes. It works alongside inheritance and method overriding. Polymorphism is one of the most powerful features of OOP.

Think of it like a command 'start': start a car means turn the ignition; start a computer means press the power button. Same command, different actions.

7.2 Animal Example

```
<?php
class Animal {
    public function __construct(public string $name) {}
    public function speak(): string { return 'Some sound'; }
    public function describe(): string { return "I am {$this->name}"; }
}

class Dog  extends Animal { public function speak(): string { return 'Woof!'; } }
class Cat  extends Animal { public function speak(): string { return 'Meow!'; } }
class Duck extends Animal { public function speak(): string { return 'Quack!'; } }
class Cow  extends Animal { public function speak(): string { return 'Moo!'; } }

// Polymorphism: same method call, different outputs
$animals = [
    new Dog("Rex"), new Cat("Whiskers"), new Duck("Donald"), new Cow("Bessie")
];

foreach ($animals as $animal) {
    echo $animal->describe() . " — " . $animal->speak() . "\n";
}
// Output:
// I am Rex — Woof!
// I am Whiskers — Meow!
// I am Donald — Quack!
// I am Bessie — Moo!
?>
```

7.3 Payment System Example

A common real-world example of polymorphism is a payment system where different payment methods share a common interface:

```
<?php
// Abstract Base Class
```

```
abstract class PaymentMethod {
    protected string $currency = 'USD';

    abstract public function processPayment(float $amount): string;
    abstract public function getMethodName(): string;

    public function formatAmount(float $amount): string {
        return "{$this->currency} " . number_format($amount, 2);
    }
}

// PayPal
class PayPalPayment extends PaymentMethod {
    private string $paypalEmail;

    public function __construct(string $paypalEmail) {
        $this->paypalEmail = $paypalEmail;
    }

    public function processPayment(float $amount): string {
        return "PayPal: {$this->formatAmount($amount)} sent from {$this-
>paypalEmail}";
    }

    public function getMethodName(): string { return "PayPal"; }
}

// Visa Card
class VisaPayment extends PaymentMethod {
    private string $cardNumber;

    public function __construct(string $cardNumber) {
        $this->cardNumber = '****' . substr($cardNumber, -4);
    }

    public function processPayment(float $amount): string {
        return "Visa Card {$this->cardNumber}: charged {$this-
>formatAmount($amount)}";
    }

    public function getMethodName(): string { return "Visa"; }
}

// Mobile Money
class MobileMoneyPayment extends PaymentMethod {
    public function __construct(private string $phoneNumber) {}

    public function processPayment(float $amount): string {
        return "M-Pesa/TigoPesa: {$this->formatAmount($amount)} via {$this-
>phoneNumber}";
```

```
    }

    public function getMethodName(): string { return "Mobile Money"; }
}

// Polymorphic Usage
function checkout(PaymentMethod $method, float $total): void {
    echo "[{$method->getMethodName()}] " . $method->processPayment($total) . "\n";
}

$paypal  = new PayPalPayment("buyer@email.com");
$visa    = new VisaPayment("4111111111111234");
$mobile  = new MobileMoneyPayment("+255712345678");

checkout($paypal, 149.99);
checkout($visa,   249.50);
checkout($mobile, 50.00);
?>
```

7.4 Interfaces

An interface defines a contract — a list of methods that any implementing class must provide. This is another form of polymorphism in PHP:

```
<?php
interface Printable {
    public function print(): string;
    public function getFormat(): string;
}

class Invoice implements Printable {
    public function __construct(private string $invoiceNumber) {}
    public function print(): string  { return "Invoice #{$this->invoiceNumber}"; }
    public function getFormat(): string { return "PDF"; }
}

class Report implements Printable {
    public function __construct(private string $title) {}
    public function print(): string  { return "Report: {$this->title}"; }
    public function getFormat(): string { return "DOCX"; }
}

$docs = [new Invoice("INV-001"), new Report("Monthly Sales")];
foreach ($docs as $doc) {
    echo "[{$doc->getFormat()}] " . $doc->print() . "\n";
}
?>
```

📋 Chapter Summary

- Polymorphism means the same method name behaves differently in different classes.
- It is achieved through method overriding (child redefines parent method).
- Abstract classes define method signatures that child classes must implement.
- Interfaces define contracts — all implementing classes must provide those methods.
- Polymorphism makes code flexible: you can process different types uniformly.

Chapter 8: OOP Database Connection

8.1 Creating the Database

Before connecting in PHP, create the database in MySQL using phpMyAdmin or the terminal:

```
-- Run in phpMyAdmin or MySQL console
CREATE DATABASE school_db;
USE school_db;

CREATE TABLE students (
    id      INT AUTO_INCREMENT PRIMARY KEY,
    name    VARCHAR(100) NOT NULL,
    email   VARCHAR(100) UNIQUE NOT NULL,
    phone   VARCHAR(20),
    course  VARCHAR(100) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

8.2 Database Class Using MySQLi

```
<?php
// File: classes/Database.php

class Database {
    private static ?Database $instance = null;  // Singleton
    private mysqli $connection;

    private string $host     = 'localhost';
    private string $username = 'root';
    private string $password = '';
    private string $database = 'school_db';

    // Private constructor — use getInstance()
    private function __construct() {
        $this->connection = new mysqli(
            $this->host, $this->username,
            $this->password, $this->database
        );

        if ($this->connection->connect_error) {
            die("Connection Error: " . $this->connection->connect_error);
        }
        $this->connection->set_charset('utf8mb4');
    }

    // Singleton pattern — only one instance allowed
```

```
    public static function getInstance(): self {
        if (self::$instance === null) {
            self::$instance = new self();
        }
        return self::$instance;
    }

    // Execute a query safely
    public function query(string $sql): mysqli_result|bool {
        return $this->connection->query($sql);
    }

    // Prepared statement (prevents SQL injection)
    public function prepare(string $sql): mysqli_stmt|false {
        return $this->connection->prepare($sql);
    }

    // Get last inserted ID
    public function lastInsertId(): int {
        return $this->connection->insert_id;
    }

    // Close connection
    public function __destruct() {
        $this->connection->close();
    }

    // Prevent cloning
    private function __clone() {}
}
?>
```

8.3 Database Class Using PDO

```
<?php
// File: classes/DatabasePDO.php

class DatabasePDO {
    private PDO $pdo;

    public function __construct(
        string $host     = "localhost",
        string $dbname   = "school_db",
        string $username = "root",
        string $password = ""
    ) {
        $dsn = "mysql:host={$host};dbname={$dbname};charset=utf8mb4";
```

```
        $options = [
            PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES   => false,
        ];

        try {
            $this->pdo = new PDO($dsn, $username, $password, $options);
        } catch (PDOException $e) {
            die("PDO Connection Error: " . $e->getMessage());
        }
    }

    // Execute a prepared statement
    public function run(string $sql, array $params = []): PDOStatement {
        $stmt = $this->pdo->prepare($sql);
        $stmt->execute($params);
        return $stmt;
    }

    // Fetch all rows
    public function fetchAll(string $sql, array $params = []): array {
        return $this->run($sql, $params)->fetchAll();
    }

    // Fetch a single row
    public function fetch(string $sql, array $params = []): mixed {
        return $this->run($sql, $params)->fetch();
    }

    // Last inserted ID
    public function lastInsertId(): string {
        return $this->pdo->lastInsertId();
    }
}
?>
```

MySQLi	PDO
MySQL only	Supports 12+ databases
Object or procedural style	Object-oriented only
Slightly faster for MySQL	More portable across databases
Good for simple MySQL projects	Recommended for production apps

📋 **Chapter Summary**

- The Database class wraps MySQLi or PDO in an OOP interface.

- Always use prepared statements to prevent SQL injection attacks.
- The Singleton pattern ensures only one database connection exists.
- PDO is more portable (works with MySQL, PostgreSQL, SQLite, etc.).
- Always set charset to utf8mb4 for proper Unicode support.

Chapter 9: Form Handling Using OOP

9.1 Student Registration Form

This example shows a complete registration flow: HTML form, OOP validation, and data processing.

HTML Form (register.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Student Registration</title>
    <link rel="stylesheet" href="assets/css/style.css">
</head>
<body>
    <div class="container">
        <h2>Student Registration</h2>
        <form action="process_register.php" method="POST">
            <label>Full Name:</label>
            <input type="text" name="name" required>

            <label>Email Address:</label>
            <input type="email" name="email" required>

            <label>Phone Number:</label>
            <input type="tel" name="phone">

            <label>Course:</label>
            <select name="course" required>
                <option value="">-- Select Course --</option>
                <option value="Computer Science">Computer Science</option>
                <option value="Information Technology">Information
Technology</option>
                <option value="Software Engineering">Software Engineering</option>
                <option value="Data Science">Data Science</option>
            </select>

            <label>Password:</label>
            <input type="password" name="password" required>

            <button type="submit" class="btn">Register</button>
        </form>
    </div>
</body>
</html>
```

Validator Class (classes/Validator.php)

```
<?php
class Validator {
    private array $errors = [];

    public function required(string $value, string $field): void {
        if (empty(trim($value))) {
            $this->errors[$field] = "{$field} is required.";
        }
    }

    public function email(string $value, string $field): void {
        if (!filter_var($value, FILTER_VALIDATE_EMAIL)) {
            $this->errors[$field] = "Invalid email format.";
        }
    }

    public function minLength(string $value, string $field, int $min): void {
        if (strlen($value) < $min) {
            $this->errors[$field] = "{$field} must be at least {$min} characters.";
        }
    }

    public function phone(string $value, string $field): void {
        if (!preg_match('/^[+]?[0-9]{10,15}$/', $value)) {
            $this->errors[$field] = "Invalid phone number.";
        }
    }

    public function isValid(): bool  { return empty($this->errors); }
    public function getErrors(): array { return $this->errors; }

    public static function sanitize(string $input): string {
        return htmlspecialchars(strip_tags(trim($input)));
    }
}
?>
```

Processing Script (process_register.php)

```
<?php
require_once 'classes/Database.php';
require_once 'classes/Validator.php';

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    header('Location: register.html');
    exit;
}
```

```
// Sanitize inputs
$name     = Validator::sanitize($_POST['name']     ?? '');
$email    = Validator::sanitize($_POST['email']    ?? '');
$phone    = Validator::sanitize($_POST['phone']    ?? '');
$course   = Validator::sanitize($_POST['course']   ?? '');
$password = $_POST['password'] ?? '';

// Validate
$v = new Validator();
$v->required($name,   'name');
$v->required($email,  'email');
$v->email($email,     'email');
$v->required($course, 'course');
$v->required($password, 'password');
$v->minLength($password, 'password', 8);

if (!$v->isValid()) {
    foreach ($v->getErrors() as $err) {
        echo "<p style='color:red'>{$err}</p>";
    }
    exit;
}

// Insert into database using prepared statement
$db   = Database::getInstance();
$hash = password_hash($password, PASSWORD_BCRYPT);

$stmt = $db->prepare("INSERT INTO students (name, email, phone, course, password)
VALUES (?,?,?,?,?)");
$stmt->bind_param('sssss', $name, $email, $phone, $course, $hash);

if ($stmt->execute()) {
    echo "<p style='color:green'>Registration successful! ID: " . $db-
>lastInsertId() . "</p>";
} else {
    echo "<p style='color:red'>Error: could not register.</p>";
}
?>
```

📋 Chapter Summary

- Always validate and sanitize user input before processing.
- Use a dedicated Validator class to separate validation logic from business logic.
- Use prepared statements (bind_param) to prevent SQL injection.
- htmlspecialchars() and strip_tags() protect against XSS attacks.
- Use POST method for form submissions that change data.

Chapter 10: CRUD Application Using OOP

10.1 Project: Student Management System

We will build a complete CRUD (Create, Read, Update, Delete) system for managing students. The project uses OOP principles and the Database class from Chapter 8.

10.2 Student Model Class

```
<?php
// File: classes/Student.php
require_once __DIR__ . '/Database.php';

class Student {
    private Database $db;

    public function __construct() {
        $this->db = Database::getInstance();
    }

    // CREATE – Insert new student
    public function create(string $name, string $email, string $phone, string
$course): bool {
        $stmt = $this->db->prepare("INSERT INTO students (name,email,phone,course)
VALUES (?,?,?,?)");
        $stmt->bind_param('ssss', $name, $email, $phone, $course);
        return $stmt->execute();
    }

    // READ – Get all students
    public function getAll(): array {
        $result = $this->db->query("SELECT * FROM students ORDER BY name ASC");
        return $result->fetch_all(MYSQLI_ASSOC);
    }

    // READ – Get one student by ID
    public function getById(int $id): array|false {
        $stmt = $this->db->prepare("SELECT * FROM students WHERE id = ?");
        $stmt->bind_param('i', $id);
        $stmt->execute();
        return $stmt->get_result()->fetch_assoc();
    }

    // UPDATE – Edit student details
    public function update(int $id, string $name, string $email, string $phone,
string $course): bool {
        $stmt = $this->db->prepare("UPDATE students SET
name=?,email=?,phone=?,course=? WHERE id=?");
        $stmt->bind_param('ssssi', $name, $email, $phone, $course, $id);
```

```
        return $stmt->execute();
    }

    // DELETE — Remove student
    public function delete(int $id): bool {
        $stmt = $this->db->prepare("DELETE FROM students WHERE id = ?");
        $stmt->bind_param('i', $id);
        return $stmt->execute();
    }

    // SEARCH — Find students by name
    public function search(string $term): array {
        $like = "%{$term}%";
        $stmt = $this->db->prepare("SELECT * FROM students WHERE name LIKE ? OR
email LIKE ?");
        $stmt->bind_param('ss', $like, $like);
        $stmt->execute();
        return $stmt->get_result()->fetch_all(MYSQLI_ASSOC);
    }
}
?>
```

10.3 index.php — List All Students

```
<?php
require_once 'classes/Student.php';
$studentModel = new Student();

$search = $_GET['q'] ?? '';
$students = $search ? $studentModel->search($search) : $studentModel->getAll();
?>
<!DOCTYPE html>
<html><head><title>Students</title></head><body>
<h1>Student Management System</h1>

<form method="GET">
    <input type="text" name="q" value="<?= htmlspecialchars($search) ?>"
placeholder="Search...">
    <button type="submit">Search</button>
</form>

<a href="add.php">+ Add New Student</a>

<table border='1' cellpadding='8'>
    <tr><th>ID</th><th>Name</th><th>Email</th><th>Course</th><th>Actions</th></tr>
    <?php foreach ($students as $s): ?>
    <tr>
        <td><?= $s['id'] ?></td>
```

```
        <td><?= htmlspecialchars($s['name']) ?></td>
        <td><?= htmlspecialchars($s['email']) ?></td>
        <td><?= htmlspecialchars($s['course']) ?></td>
        <td>
            <a href='edit.php?id=<?= $s['id'] ?>'>Edit</a> |
            <a href='delete.php?id=<?= $s['id'] ?>'
               onclick='return confirm("Delete this student?")'>Delete</a>
        </td>
    </tr>
    <?php endforeach; ?>
</table>
</body></html>
```

10.4 add.php — Create Student

```
<?php
require_once 'classes/Student.php';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $model = new Student();
    $ok = $model->create(
        htmlspecialchars($_POST['name']),
        htmlspecialchars($_POST['email']),
        htmlspecialchars($_POST['phone']),
        htmlspecialchars($_POST['course'])
    );
    header($ok ? 'Location: index.php' : 'Location: add.php?error=1');
    exit;
}
?>
<form method="POST">
    <input name="name"   placeholder="Full Name"   required>
    <input name="email"  placeholder="Email"        required type="email">
    <input name="phone"  placeholder="Phone">
    <input name="course" placeholder="Course"       required>
    <button type="submit">Save</button>
</form>
```

10.5 edit.php — Update Student

```
<?php
require_once 'classes/Student.php';
$model = new Student();
$id = (int)($_GET['id'] ?? 0);
$student = $model->getById($id);

if (!$student) { echo 'Not found'; exit; }
```

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $model->update($id,
        htmlspecialchars($_POST['name']),
        htmlspecialchars($_POST['email']),
        htmlspecialchars($_POST['phone']),
        htmlspecialchars($_POST['course'])
    );
    header('Location: index.php'); exit;
}
?>
<form method="POST">
    <input name="name"   value="<?= htmlspecialchars($student['name']) ?>">
    <input name="email"  value="<?= htmlspecialchars($student['email']) ?>">
    <input name="phone"  value="<?= htmlspecialchars($student['phone']) ?>">
    <input name="course" value="<?= htmlspecialchars($student['course']) ?>">
    <button type="submit">Update</button>
</form>
```

10.6 delete.php — Delete Student

```
<?php
require_once 'classes/Student.php';
$model = new Student();
$id = (int)($_GET['id'] ?? 0);

if ($id > 0 && $model->delete($id)) {
    header('Location: index.php');
} else {
    echo 'Delete failed or invalid ID.';
}
exit;
?>
```

📋 **Chapter Summary**

- CRUD = Create, Read, Update, Delete — the four basic database operations.
- The Student class (model) encapsulates all database interactions.
- Always use prepared statements to prevent SQL injection.
- Separate concerns: model handles data, PHP view handles display.
- htmlspecialchars() prevents XSS when outputting user data in HTML.

Chapter 11: Project Structure in VS Code

11.1 Recommended Folder Structure

```
student-management/
├── classes/
│   ├── Database.php        # Database connection (MySQLi or PDO)
│   ├── Student.php         # Student model (CRUD operations)
│   ├── User.php            # User authentication model
│   └── Validator.php       # Input validation logic
├── config/
│   └── config.php          # Global settings (DB creds, constants)
├── public/
│   ├── index.php           # Student list page
│   ├── add.php             # Add student form
│   ├── edit.php            # Edit student form
│   ├── delete.php          # Delete handler
│   ├── register.php        # User registration
│   └── login.php           # User login
├── assets/
│   ├── css/
│   │   └── style.css       # Main stylesheet
│   └── js/
│       └── main.js         # JavaScript (optional)
├── sql/
│   └── school_db.sql       # Database export/backup
└── index.php               # Root redirect to public/index.php
```

11.2 Folder Responsibilities

Folder/File	Purpose
classes/	Contains all OOP class files (models, helpers)
config/	Global configuration: DB credentials, app settings
public/	Web-accessible PHP pages (controller + view)
assets/	Static files: CSS, JS, images
sql/	SQL schema and seed files for the database
index.php	Root entry point, usually redirects to public/

11.3 config/config.php

```
<?php
// File: config/config.php
// Central configuration file
```

```
define('DB_HOST',     'localhost');
define('DB_USER',     'root');
define('DB_PASS',     '');
define('DB_NAME',     'school_db');
define('APP_NAME',    'Student Management System');
define('BASE_URL',    'http://localhost/student-management/');
define('ITEMS_PER_PAGE', 10);

// Autoloader — automatically includes class files
spl_autoload_register(function (string $class): void {
    $file = __DIR__ . "/../classes/{$class}.php";
    if (file_exists($file)) {
        require_once $file;
    }
});
?>
```

📋 Chapter Summary

- A clean folder structure separates concerns: models, views, config, assets.
- Classes go in /classes/, config in /config/, web pages in /public/.
- Use spl_autoload_register() to avoid writing require_once for every class.
- Store DB credentials in a config file, never hardcode them in class files.

Chapter 12: Mini Project — Student Registration & Management System

12.1 Project Overview

Project Specifications

Name: Student Registration and Management System

Stack: PHP OOP + MySQL + Bootstrap CSS

Server: XAMPP / LAMPP

Features: Registration, Login, Add/Edit/Delete/Search Student, Logout

12.2 Database Schema

```
-- File: sql/school_db.sql
CREATE DATABASE IF NOT EXISTS school_db;
USE school_db;

CREATE TABLE users (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    username   VARCHAR(50) UNIQUE NOT NULL,
    email      VARCHAR(100) UNIQUE NOT NULL,
    password   VARCHAR(255) NOT NULL,
    role       ENUM('admin','staff') DEFAULT 'staff',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE students (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    email      VARCHAR(100) UNIQUE NOT NULL,
    phone      VARCHAR(20),
    course     VARCHAR(100) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Sample admin user (password: admin1234)
INSERT INTO users (username, email, password, role) VALUES
('admin', 'admin@school.com', '$2y$10$...hashed...', 'admin');
```

12.3 Auth Class

```
<?php
```

```
// File: classes/Auth.php

class Auth {
    private Database $db;

    public function __construct() {
        $this->db = Database::getInstance();
        if (session_status() === PHP_SESSION_NONE) { session_start(); }
    }

    public function login(string $username, string $password): bool {
        $stmt = $this->db->prepare("SELECT * FROM users WHERE username = ? LIMIT
1");
        $stmt->bind_param('s', $username);
        $stmt->execute();
        $user = $stmt->get_result()->fetch_assoc();

        if ($user && password_verify($password, $user['password'])) {
            $_SESSION['user_id']   = $user['id'];
            $_SESSION['username']  = $user['username'];
            $_SESSION['role']      = $user['role'];
            return true;
        }
        return false;
    }

    public function logout(): void {
        session_destroy();
        header('Location: login.php');
        exit;
    }

    public function isLoggedIn(): bool {
        return isset($_SESSION['user_id']);
    }

    public function requireLogin(): void {
        if (!$this->isLoggedIn()) {
            header('Location: login.php'); exit;
        }
    }
}
?>
```

12.4 Login Page (login.php)

```
<?php
require_once '../config/config.php';
```

```
$auth = new Auth();

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $ok = $auth->login($_POST['username'], $_POST['password']);
    if ($ok) { header('Location: index.php'); exit; }
    $error = 'Invalid username or password.';
}
?>
<!DOCTYPE html>
<html><head><title>Login</title></head><body>
<?php if (isset($error)): ?>
    <p style="color:red"><?= $error ?></p>
<?php endif; ?>
<form method="POST">
    <input type="text"     name="username" placeholder="Username" required>
    <input type="password" name="password" placeholder="Password" required>
    <button type="submit">Login</button>
</form>
</body></html>
```

12.5 Dashboard (index.php)

```
<?php
require_once '../config/config.php';
$auth = new Auth();
$auth->requireLogin();  // Redirect to login if not authenticated

$studentModel = new Student();
$q        = $_GET['q'] ?? '';
$students = $q ? $studentModel->search($q) : $studentModel->getAll();
?>
<!DOCTYPE html>
<html><head><title>Dashboard</title></head><body>
<h1>Welcome, <?= htmlspecialchars($_SESSION["username"]) ?>!</h1>
<a href="logout.php">Logout</a>  |  <a href="add.php">+ Add Student</a>

<!-- Search form -->
<form method="GET">
    <input name="q" value="<?= htmlspecialchars($q) ?>" placeholder="Search name or
email...">
    <button>Search</button>
</form>

<!-- Student table -->
<table border='1'>
<tr><th>#</th><th>Name</th><th>Email</th><th>Course</th><th>Actions</th></tr>
<?php foreach ($students as $s): ?>
<tr>
    <td><?= $s['id'] ?></td>
```

```
    <td><?= htmlspecialchars($s['name']) ?></td>
    <td><?= htmlspecialchars($s['email']) ?></td>
    <td><?= htmlspecialchars($s['course']) ?></td>
    <td>
        <a href='edit.php?id=<?= $s["id"] ?>'>Edit</a> |
        <a href='delete.php?id=<?= $s["id"] ?>'
           onclick='return confirm("Sure?")'>Delete</a>
    </td>
</tr>
<?php endforeach; ?>
</table>
</body></html>
```

📋 Chapter Summary

- The mini project combines all OOP concepts: Classes, Inheritance, Encapsulation.
- Auth class handles login, logout, and session management.
- Always call requireLogin() at the top of protected pages.
- Separate concerns: Auth handles security, Student handles data, pages handle display.
- This project demonstrates a real-world PHP OOP web application.

Chapter 13: Practical Exercises

Exercise 1: Book Class

✏️ **Exercise: Book Class**

1. Create a Book class with: title, author, ISBN, price, year.

2. Add a constructor that initializes all properties.

3. Add methods: getSummary(), applyDiscount(float $percent), isAffordable(float $budget).

4. Create 5 Book objects and display all their summaries.

5. Use a loop to find all books under $30.

Exercise 2: Car Class with Inheritance

✏️ **Exercise: Car Inheritance**

1. Create a Vehicle parent class with: brand, model, year, fuelType.

2. Create a Car child class that extends Vehicle, adding: doors, transmission.

3. Create an ElectricCar child class of Car, adding: batteryCapacity, range.

4. Override a describe() method in each class to show class-specific details.

5. Demonstrate polymorphism by storing all vehicle types in one array.

Exercise 3: User System with Encapsulation

✏️ **Exercise: User Encapsulation**

1. Create a User class with private: username, email, passwordHash, loginAttempts.

2. Add setters that validate: email format, password strength (8+ chars, 1 digit).

3. Add methods: login(password), resetAttempts(), lockAccount() after 3 failed attempts.

4. Add a getter for username and email only — never expose the password hash.

5. Test with correct and incorrect passwords, verify lockout after 3 failures.

Exercise 4: Database Connection Class

✏️ **Exercise: Database OOP**

1. Implement the Database class from Chapter 8 using the Singleton pattern.

2. Add a countRows(string $table) method that returns the row count.

3. Add a tableExists(string $table) method that checks if a table exists.

4. Test with your school_db database.

5. Verify only one instance is created even if getInstance() is called 10 times.

Exercise 5: Shape Polymorphism

✏️ **Exercise: Shape Polymorphism**

1. Create an abstract Shape class with abstract methods: area() and perimeter().

2. Create Circle, Rectangle, and Triangle classes extending Shape.

3. Each class implements area() and perimeter() with correct formulas.

4. Create an array of 6 mixed shapes and print their areas and perimeters.

5. Add a method to find the shape with the largest area.

Exercise 6: Student Registration Form

✏️ **Exercise: Full Registration Form**

1. Build a complete HTML form for student registration (name, email, phone, course, password).

2. Use the Validator class to validate all fields before saving.

3. Display specific error messages next to each field on validation failure.

4. On success, save to the students table and redirect to a success page.

5. Protect against SQL injection using prepared statements.

Chapter 14: Solved Assignments

Assignment 1: Library Management System

Objectives
Build a PHP OOP system to manage library books and borrowers.

Requirements:

- Classes: Book, Member, Loan
- Database tables: books (id, title, author, isbn, quantity), members (id, name, email), loans (id, book_id, member_id, due_date, returned)
- Features: Add/search books, register members, borrow/return books, view overdue loans

Expected Output:

- Member can borrow a book quantity decreases by 1
- Return book quantity increases, loan marked returned
- Search books by title or author
- List of all overdue loans

Assignment 2: Hospital Management System

Objectives
Build a PHP OOP system to manage hospital patients and appointments.

Requirements:

- Classes: Patient, Doctor, Appointment
- Database tables: patients, doctors, appointments
- Features: Register patient, add doctor, book appointment, view schedule, mark appointment complete
- Validation: future dates only, no double-booking per doctor

Assignment 3: School Management System

Objectives
Build a PHP OOP system to manage students, teachers, and classes.

Requirements:

- Classes: Student, Teacher, Classroom, Grade
- Features: Enroll student in class, assign teacher, record grades, generate report card
- Calculate GPA from a student's grades
- List students by course or by teacher

Assignment 4: Inventory Management System

Objectives
Build a PHP OOP system to manage product inventory.

Requirements:

- Classes: Product, Category, StockTransaction
- Features: Add products, record stock-in/stock-out transactions, generate low-stock alerts
- Filter products by category, price range, or stock level
- Generate a simple inventory report showing total value

Assignment 5: Employee Management System

Objectives
Build a PHP OOP system for managing company employees.

Requirements:

- Classes: Employee, Department, Salary, Leave
- Inheritance: FullTimeEmployee and PartTimeEmployee extend Employee
- Features: Add employee, assign department, record salary, request/approve leave
- Calculate monthly payroll report per department

Chapter 15: PHP OOP Interview Questions & Answers

The following 30+ questions are commonly asked in PHP developer interviews. Study these carefully!

Q1: What is Object-Oriented Programming (OOP)?

Answer: OOP is a programming paradigm that organizes code into classes and objects. It is based on four pillars: Encapsulation, Inheritance, Polymorphism, and Abstraction. OOP promotes code reuse, modularity, and easier maintenance.

Q2: What is a class in PHP?

Answer: A class is a blueprint or template that defines properties (variables) and methods (functions) for creating objects. It does not use memory on its own. Example: class Car { public string $brand; public function drive() { ... } }

Q3: What is an object?

Answer: An object is an instance of a class a concrete copy of the blueprint with its own data. Example: $car = new Car();

Q4: What is __construct()?

Answer: __construct() is a magic method automatically called when an object is created with new. It initializes the object's properties.

Q5: What is __destruct()?

Answer: __destruct() is called automatically when an object is destroyed or the script ends. Used to clean up resources like database connections and file handles.

Q6: What is encapsulation?

Answer: Encapsulation is hiding an object's internal data (properties) and exposing only necessary functionality through public methods (getters/setters). It protects data from accidental modification.

Q7: What are the access modifiers in PHP?

Answer: PHP has three: public (accessible everywhere), protected (accessible in the class and subclasses), and private (accessible only within the defining class).

Q8: What is inheritance?

Answer: Inheritance allows a child class to acquire properties and methods from a parent class using the extends keyword. It promotes code reuse.

Q9: What is the parent:: keyword?

Answer: parent:: is used in child classes to call the parent class's constructor or methods. Example: parent::__construct($name);

Q10: What is polymorphism?

Answer: Polymorphism means 'many forms.' The same method name can behave differently in different classes. Achieved through method overriding.

Q11: What is method overriding?

Answer: Method overriding is when a child class defines a method with the same name as a parent class method. The child's version replaces the parent's for objects of that child class.

Q12: What is an abstract class?

Answer: An abstract class cannot be instantiated. It can define abstract methods (signature only) that child classes must implement. Declared with: abstract class Shape { abstract public function area(): float; }

Q13: What is an interface?

Answer: An interface defines a contract a list of method signatures that implementing classes must provide. Use interface Printable { public function print(): string; } and class Invoice implements Printable { ... }

Q14: Difference between abstract class and interface?

Answer: Abstract class: can have implemented methods, properties, and constructors; single inheritance. Interface: only method signatures (in PHP 8, can have constants); supports multiple implementation.

Q15: What is the $this keyword?

Answer: $this refers to the current object inside a class method. Used to access the object's own properties and methods.

Q16: What is the self:: keyword?

Answer: self:: refers to the current class itself (not the object). Used to access static properties and methods. Example: self::$count++; or self::$instance = new self();

Q17: What is a static method/property?

Answer: Static members belong to the class, not individual objects. They are accessed with ClassName::member or self::member. Useful for counters, singletons, and utility functions.

Q18: What is the Singleton pattern?

Answer: Singleton ensures only one instance of a class exists. Achieved by making the constructor private and providing a static getInstance() method. Used for database connections.

Q19: What is the final keyword?

Answer: final on a class prevents it from being extended. final on a method prevents it from being overridden.

Q20: What is constructor property promotion (PHP 8)?

Answer: A PHP 8 shorthand: declare and initialize properties directly in the constructor parameter list. Example: public function __construct(public string $name, public int $age) {}

Q21: What is PDO?

Answer: PDO (PHP Data Objects) is a database abstraction layer that supports 12+ databases (MySQL, PostgreSQL, SQLite, etc.). It uses prepared statements, is object-oriented, and is recommended for production.

Q22: What is the difference between MySQLi and PDO?

Answer: MySQLi supports only MySQL and can be procedural or OOP. PDO supports 12+ databases and is OOP-only. PDO is more portable; MySQLi is simpler for MySQL-only projects.

Q23: What is a prepared statement and why use it?

Answer: A prepared statement separates SQL from data using placeholders (?). PHP sends them separately to MySQL, making SQL injection impossible. Always use for user-supplied data.

Q24: What is SQL injection?

Answer: SQL injection is when a malicious user inserts SQL code into user input to manipulate the database. Example: entering ' OR 1=1 -- in a login form. Prevention: always use prepared statements.

Q25: What is the difference between == and === in PHP?

Answer: == is loose comparison (converts types): '1' == 1 is true. === is strict comparison (same type AND value): '1' === 1 is false.

Q26: What is the difference between include and require?

Answer: Both include files. include: on failure, shows a warning and continues. require: on failure, stops the script with a fatal error. Use require for critical files like Database.php.

Q27: What is namespace in PHP?

Answer: Namespaces organize code to avoid name collisions between classes from different packages. Example: namespace App\Models; class Student {} — accessed as App\Models\Student.

Q28: What is a trait in PHP?

Answer: A trait is a way to share methods between classes that don't have an inheritance relationship. PHP supports single inheritance, but a class can use multiple traits.

Q29: What is the difference between GET and POST?

Answer: GET appends data to the URL (visible, bookmarkable, not for sensitive data). POST sends data in the request body (hidden, not cached). Use POST for forms that change data.

Q30: What are magic methods in PHP?

Answer: Magic methods start with __ and are automatically called by PHP. Common: __construct(), __destruct(), __toString(), __get(), __set(), __call(), __clone(), __isset().

Q31: What is type hinting?

Answer: Type hinting declares the expected type for method parameters and return values. Example: public function add(int $a, int $b): int { return $a + $b; } PHP will throw a TypeError if wrong type is passed.

Q32: What is exception handling in PHP OOP?

Answer: Exceptions handle errors gracefully using try-catch-finally blocks. Throw an exception: throw new Exception('Error'); Catch it: catch (Exception $e) { echo $e->getMessage(); }

Quick Reference Card

Concept	PHP Syntax
Define a class	class Student { ... }
Create an object	$s = new Student('Alice', 20);
Access property	$s->name
Call method	$s->displayInfo()
Constructor	public function __construct(...) { }
Destructor	public function __destruct() { }
Inheritance	class Child extends Parent { }
Call parent constructor	parent::__construct(...)
Override method	Same method name in child class
Abstract class	abstract class Shape { abstract public function area(): float; }
Interface	interface Printable { public function print(): string; }
Implement interface	class Doc implements Printable { }
Static method	public static function count(): int { }
Call static method	ClassName::count()
self:: vs $this	self:: = class, $this = object
Final class	final class Sealed { }
Final method	final public function lock(): void { }
Trait	trait Loggable { public function log() {...} }
Use trait	class User { use Loggable; }
Type hinting (PHP 8)	public function add(int $a, int $b): int { }
Prepared stmt (MySQLi)	$stmt = $db->prepare('SELECT * WHERE id=?'); $stmt->bind_param('i',$id);
Prepared stmt (PDO)	$stmt = $pdo->prepare('SELECT...'); $stmt->execute([$id]);